SNOMED CT MCP Tools on InterSystems IRIS for Health

Development Team Handoff Document

Generated May 18, 2026 from 15 uploaded planning / implementation story documents.

Purpose: consolidate story-level planning artifacts into one coherent engineering handoff. This PDF preserves implementation decisions, acceptance criteria, validation notes, and file-level deliverables while organizing them into a development sequence.

1. Executive Summary

This project delivers a SNOMED CT terminology service for InterSystems IRIS for Health AI Hub. It exposes concept lookup, lexical search, hierarchy navigation, and semantic vector search as MCP tools usable from Claude Desktop/Code or another MCP client.

Primary outcomes

- RF2 concept, description, and relationship data can be loaded into persistent IRIS SQL tables.
- Full-text lexical search is available through IRIS iFind.
- SNOMED IS-A hierarchy is precomputed into globals for fast traversal and subsumption checks.
- MCP tools are wrapped with %AI.Tool, grouped by %AI.ToolSet, and served by %AI.MCP.Service.
- Semantic search uses IRIS vector storage and Ollama embeddings.
- IPM and Docker packaging support repeatable deployment.

Delivery status

- All 15 uploaded stories are marked Status: review.
- Fixture-based validation is recorded for most data, search, hierarchy, vector, and tool paths.
- Full UK Monolith load timing and full Docker image build require follow-up validation in the target environment.

2. Target Architecture

```
Claude Desktop / Claude Code
    -> iris-mcp-server (stdio)
        -> IRIS endpoint /mcp/snomed
            -> SNOMED.MCP.Service (%AI.MCP.Service)
                -> SNOMED.ToolSet (%AI.ToolSet)
                    -> Lookup / Search / Hierarchy / Semantic MCP tools
                        -> IRIS SQL tables, globals, iFind index, VECTOR/HNSW index, Ollama
```

Layer	Classes / files	Responsibility
Data model	SNOMED.Concept; SNOMED.Description; SNOMED.Relationship; SNOMED.ConceptEmbedding	Persistent storage for RF2 terminology and vectors.
Data ingestion	SNOMED.Loader	Embedded Python RF2 loaders using streamed TSV parsing and prepared SQL inserts.
Search / hierarchy core	SNOMED.Search; SNOMED.TransitiveClosure; SNOMED.Hierarchy	Lexical search, vector search, transitive closure computation, hierarchy traversal.
MCP wrappers	SNOMED.Tools.Lookup; Search; Hierarchy; Semantic	LLM-facing methods with JSON results and doc comments for discovery.
MCP service	SNOMED.ToolSet; SNOMED.MCP.Service; config/mcp-config.toml	Tool registration, service endpoint, iris-mcp-server connectivity.
Packaging	module.xml; SNOMED.Installer; Dockerfile; docker-compose.yml; README.md	Install, configure, run, and document the application.

3. Implementation Roadmap

Phase 1 - RF2 data foundation

Implement persistent schemas and streaming loaders for concepts, descriptions, and relationships. All SCTIDs are strings, never numeric. Loaders use Embedded Python and SQL prepared statements.

Phase 2 - Search and hierarchy core

Add iFind lexical search on description terms, compute transitive closure for IS-A relationships into globals, and implement ObjectScript hierarchy navigation methods.

Phase 3 - MCP exposure

Wrap lookup, lexical search, pattern search, and hierarchy navigation as %AI.Tool classes, then register through SNOMED.ToolSet and SNOMED.MCP.Service.

Phase 4 - Semantic search

Add vector storage with a 384-dimension VECTOR(DOUBLE) property and HNSW index, generate embeddings via Ollama, and expose semantic search as an MCP tool.

Phase 5 - Packaging and onboarding

Create IPM package metadata, installer, Docker image scaffolding, docker-compose with Ollama sidecar, and README quick-start documentation.

4. Data and Runtime Components

Persistent schema decisions

- SCTIDs are stored as strings throughout.
- RF2 effectiveTime is stored as an 8-character string where modeled.
- Description.Term uses MAXLEN 1024 to accommodate long FSNs.
- ConceptEmbedding uses 384-dimensional DOUBLE vectors to match all-minilm / common Ollama embedding output.

Runtime storage decisions

- ^SNOMED.TC(ancestorId, descendantId) stores all transitive IS-A paths.
- ^SNOMED.Children(parentId, childId) stores direct hierarchy edges.
- %iFind.Index.Basic powers lexical full-text search.
- %SQL.Index.HNSW powers approximate-nearest-neighbor vector search.

5. MCP Tool Surface

Tool	Class	Signature	Purpose
GetConcept	SNOMED.Tools.Lookup	GetConcept(sctid)	Lookup concept by SCTID; returns ConceptId, FSN, Active, descriptions.
LexicalSearch	SNOMED.Tools.Search	LexicalSearch(term, maxResults=20)	iFind lexical search over active descriptions/concepts.
SearchByPattern	SNOMED.Tools.Search	SearchByPattern(pattern, maxResults=50)	LIKE-based query-as-tool for wildcard matching.
IsA	SNOMED.Tools.Hierarchy	IsA(conceptId, ancestorId)	Subsumption check; JSON boolean result.

Tool	Class	Signature	Purpose
GetAncestors	SNOMED.Tools.Hierarchy	GetAncestors(conceptId)	Ancestor concepts enriched with FSN.
GetDescendants	SNOMED.Tools.Hierarchy	GetDescendants(conceptId, maxResults=100)	Descendant concepts enriched with FSN and capped.
GetChildren	SNOMED.Tools.Hierarchy	GetChildren(conceptId)	Direct child concepts enriched with FSN.
GetParents	SNOMED.Tools.Hierarchy	GetParents(conceptId)	Direct parent concepts enriched with FSN.
SemanticSearch	SNOMED.Tools.Semantic	SemanticSearch(query, maxResults=10)	Vector semantic search via Ollama and VECTOR_COSINE.

6. Validation and Known Gaps

Validated paths include class compilation, fixture RF2 loading, hierarchy globals, lexical search, vector insert/query, ToolSet discovery, and MCP service compilation/configuration.

Area	Validation status from uploaded artifacts	Follow-up
Full RF2 performance	Architecture targets under 60 seconds for UK Monolith concept/description load, but full dataset timing was not tested.	Run benchmark with licensed RF2 dataset and capture timings.
Docker build	Dockerfile and compose syntax were validated; full build was blocked by InterSystems container registry access.	Build in an environment with registry credentials and verify startup.
Ollama availability	Error paths were validated; real embedding generation depends on Ollama/model availability.	Pin or document model pull/setup and test semantic search end-to-end.
Deployment auth	Local-dev web app registration uses unauthenticated access in installer notes.	Decide production authentication and web application security settings.

7. Story Inventory

Story	Name	Status	Primary files
1-1	1-1-snomed-ct-data-model	review	src/SNOMED/Concept.cls (NEW); src/SNOMED/Description.cls (NEW); src/SNOMED/Relationship.cls (NEW)
1-2	1-2-rf2-concept-and-description-loader	review	src/SNOMED/Loader.cls (NEW); tests/fixtures/rf2/sct2_Concept_Snapshot_Test.txt (NEW); tests/fixtures/rf2/sct2_Description_Snapshot_Test.txt (NEW)
1-3	1-3-rf2-relationship-loader	review	src/SNOMED/Loader.cls (MODIFIED — added LoadRelationships, updated TruncateAll); tests/fixtures/rf2/sct2_Relationship_Snapshot_Test.txt (NEW)
2-1	2-1-full-text-lexical-search	review	src/SNOMED/Description.cls (MODIFIED — added TermIdx iFind index); src/SNOMED/Search.cls (NEW)
2-2	2-2-transitive-closure-computation	review	src/SNOMED/TransitiveClosure.cls (NEW)
2-3	2-3-hierarchy-navigation-methods	review	src/SNOMED/Hierarchy.cls (NEW)
3-1	3-1-snomed-lookup-and-search-tools	review	src/SNOMED/Tools/Lookup.cls (NEW); src/SNOMED/Tools/Search.cls (NEW)
3-2	3-2-snomed-hierarchy-tools	review	src/SNOMED/Tools/Hierarchy.cls (NEW)

Story	Name	Status	Primary files
3-3	3-3-mcp-service-and-server-configuration	review	src/SNOMED/ToolSet.cls (NEW); src/SNOMED/MCP/Service.cls (NEW); config/mcp-config.toml (NEW)
4-1	4-1-embedding-storage-and-vector-schema	review	src/SNOMED/ConceptEmbedding.cls (NEW)
4-2	4-2-embedding-generation-pipeline	review	src/SNOMED/Embeddings.cls (NEW)
4-3	4-3-semantic-search-method-and-mcp-tool	review	src/SNOMED/Search.cls (MODIFIED — added Semantic method); src/SNOMED/Tools/Semantic.cls (NEW); src/SNOMED/ToolSet.cls (MODIFIED — added Include for Semantic)
5-1	5-1-ipm-package-structure	review	module.xml (NEW); requirements.txt (NEW); src/SNOMED/Installer.cls (NEW)
5-2	5-2-docker-container-image	review	Dockerfile (NEW); docker-compose.yml (NEW); iris-init.script (NEW)
5-3	5-3-documentation-and-getting-started-guide	review	README.md (MODIFIED — added IRIS AI Hub section); config/mcp-config.toml (MODIFIED — added inline comments)

8. Story-by-Story Handoff

1-1-snomed-ct-data-model

Status: review | Source: 1-1-snomed-ct-data-model.html

User story

As a developer, I want persistent classes for SNOMED CT concepts, descriptions, and relationships with appropriate indexes, so that RF2 data can be stored and queried via SQL.

Acceptance criteria

1. SQL tables SNOMED.Concept , SNOMED.Description , and SNOMED.Relationship exist after class compilation
2. SNOMED.Concept has properties: ConceptId (string, unique indexed), Active (boolean), EffectiveTime (string), ModuleId (string), DefinitionStatusId (string)
3. SNOMED.Description has properties: DescriptionId (string, unique indexed), ConceptId (indexed), Term (string, max 1024), TypeId (string), Active (boolean), LanguageCode (string)
4. SNOMED.Relationship has properties: RelationshipId (string, unique indexed), SourceId (indexed), DestinationId (indexed), TypeId (indexed), Active (boolean), RelationshipGroup (integer)
5. All SCTID fields (ConceptId, DescriptionId, RelationshipId, SourceId, DestinationId, TypeId, ModuleId, DefinitionStatusId) are stored as %String — never numeric
6. Classes compile cleanly and SQL tables are queryable via SELECT * FROM SNOMED.Concept etc.

Implementation completion notes

- Tasks 1-3 complete: All three persistent classes created with correct properties, types, and indexes per AC #1-#5
- All SCTID fields stored as %String(MAXLEN=20) — never numeric (AC #5)
- SNOMED.Concept : ConceptId (unique), Active (bitmap), EffectiveTime, ModuleId, DefinitionStatusId
- SNOMED.Description : DescriptionId (unique), ConceptId (indexed), Term (MAXLEN=1024), TypeId (indexed), Active, LanguageCode
- SNOMED.Relationship : RelationshipId (unique), SourceId (indexed), DestinationId (indexed), TypeId (indexed), Active, RelationshipGroup
- Task 4 verified: All 3 classes compiled cleanly via $SYSTEM.OBJ.LoadDir() , SQL tables confirmed with correct columns (Concept:6, Description:7, Relationship:7), SELECT queries return SQLCODE 0
- src/SNOMED/Concept.cls (NEW)
- src/SNOMED/Description.cls (NEW)
- src/SNOMED/Relationship.cls (NEW)

Critical implementation notes

- SCTIDs are STRINGS, not numbers. SNOMED concept IDs like 138875005 look numeric but can exceed integer limits and must preserve leading zeros in some contexts. Always use %String .
- EffectiveTime is a STRING in RF2 format (YYYYMMDD). Store as %String(MAXLEN=8) , not %Date .
- TypeId fields are SCTIDs referencing other concepts (e.g., 116680003 = IS-A relationship type, 900000000000003001 = FSN type). Store as %String .
- No %New() / %Save() pattern for bulk loading — Story 1.2 will use SQL prepared statements for performance. These classes just define the schema.
- Property MAXLEN matters — Term in descriptions can be very long (up to ~1000 chars for some FSNs). Use MAXLEN = 1024 .
- Do NOT add foreign key constraints between tables — RF2 data may have dangling references during partial loads

- Do NOT add computed/derived properties — keep classes as pure data containers
- Do NOT add class methods yet — those come in later stories (Loader in 1.2, Search in 2.1, etc.)
- Do NOT create globals directly — %Persistent handles storage mapping automatically
- Do NOT use %Integer for any ID field — all IDs are strings
- Classes go in src/SNOMED/ directory following IPM conventions (Story 5.1)
- Package name is SNOMED — all classes use this prefix
- File naming: Concept.cls , Description.cls , Relationship.cls (PascalCase matching class name)
- Target namespace: USER (default IRIS development namespace)
- Platform: InterSystems IRIS for Health 2026.2.0AI (Docker container)
- Classes are compiled with $SYSTEM.OBJ.Compile("SNOMED.Concept", "ck") or via Studio/VS Code ObjectScript extension
- SQL access is automatic once classes extend %Persistent — table name = package.class
- The %Storage.Default storage strategy is used (default) — no custom storage mapping needed
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md#6. Recommended Architecture]
- [Source: _bmad-output/planning-artifacts/epics.md#Story 1.1]
- [Source: _bmad-output/project-context.md — SCTIDs are strings, RF2 is tab-separated]

Files changed

- src/SNOMED/Concept.cls (NEW)
- src/SNOMED/Description.cls (NEW)
- src/SNOMED/Relationship.cls (NEW)

1-2-rf2-concept-and-description-loader

Status: review | Source: 1-2-rf2-concept-and-description-loader.html

User story

As a terminology administrator, I want to load SNOMED CT concept and description RF2 files into IRIS, so that the terminology is available for querying.

Acceptance criteria

1. SNOMED.Loader.LoadConcepts(filepath) inserts all rows from an RF2 concept file into SNOMED.Concept via SQL prepared statements
2. SNOMED.Loader.LoadDescriptions(filepath) inserts all rows from an RF2 description file into SNOMED.Description
3. Processing is streamed line-by-line (no full dataset in memory)
4. Progress is reported every 50,000 records
5. Methods return $$$OK status on success
6. Invalid or missing filepath returns a meaningful error status
7. The UK Monolith (831K+ concepts, ~1.5M descriptions) loads in under 60 seconds total

Implementation completion notes

- Created SNOMED.Loader class with Embedded Python methods ([Language = python])
- LoadConcepts : streams RF2 concept file via csv.DictReader , inserts via iris.sql.prepare() , reports progress every 50K rows
- LoadDescriptions : streams RF2 description file, maps all 6 columns correctly (DescriptionId, ConceptId, Term, TypeId, Active, LanguageCode)
- TruncateAll : utility to clear Concept and Description tables for re-loading
- Error handling: validates filepath exists, returns %Status error with message if missing
- Verified in Docker container (iris-ai-hub): Compilation: clean, no errors LoadConcepts: 5 test concepts loaded, data verified (ConceptId=73211009, Active=1, EffectiveTime=20020131) LoadDescriptions: 5 test descriptions loaded, data verified (Term="Diabetes mellitus (disorder)", Lang="en") Error handling: missing file returns ERROR #5001 with descriptive message TruncateAll: both tables zeroed out after call
- Compilation: clean, no errors
- LoadConcepts: 5 test concepts loaded, data verified (ConceptId=73211009, Active=1, EffectiveTime=20020131)
- LoadDescriptions: 5 test descriptions loaded, data verified (Term="Diabetes mellitus (disorder)", Lang="en")
- Error handling: missing file returns ERROR #5001 with descriptive message
- TruncateAll: both tables zeroed out after call
- AC #7 (performance < 60s for UK Monolith) not tested — requires full RF2 dataset; architecture guarantees performance per research report estimates
- src/SNOMED/Loader.cls (NEW)
- tests/fixtures/rf2/sct2_Concept_Snapshot_Test.txt (NEW)
- tests/fixtures/rf2/sct2_Description_Snapshot_Test.txt (NEW)

Critical implementation notes

- Use csv.DictReader with delimiter='\t' — this handles the header row automatically and gives dict access by column name
- int(row['active']) — RF2 active field is "0" or "1" string; IRIS %Boolean expects integer 0/1

- iris.sql.prepare() is efficient — prepared statement is compiled once, executed N times. This is the recommended pattern for bulk inserts (not %New() / %Save())
- Error status pattern: Use iris.cls("%SYSTEM.Status").Error(code, message) for errors, .OK() for success
- No transaction wrapping needed — individual INSERTs auto-commit. For 831K rows this is actually faster than a single large transaction (avoids journal growth)
- Stream, don't buffer — never load the entire file into memory. csv.DictReader iterates line-by-line
- print() for progress — in Embedded Python, print() outputs to the IRIS terminal/session output
- SCTIDs are strings — pass them directly as strings to SQL parameters, never convert to int
- Python + SQL prepared statements: ~30-60 seconds for 831K concepts on modern hardware
- SQL INSERT layer dominates cost (not Python parsing)
- No need for batching or transactions — single-row INSERTs with prepared statements are optimal on IRIS
- csv.DictReader is efficient for streaming TSV — no need for custom parsing
- Do NOT use %New() / %Save() pattern — too slow for bulk loading (object overhead per row)
- Do NOT load entire file into memory (e.g., f.readlines() or csv.reader collecting to list)
- Do NOT wrap in a single transaction — journal bloat will slow things down
- Do NOT use iris.gref (global references) for inserts — SQL prepared statements are the correct approach for table inserts
- Do NOT convert SCTIDs to integers — they are strings
- Do NOT skip the header row manually — csv.DictReader handles this
- Do NOT add EffectiveTime to Description inserts — our Description schema doesn't store it (Story 1.1 decision)
- New file: src/SNOMED/Loader.cls
- Package: SNOMED (same as existing classes)
- Target namespace: USER
- Compilation: $SYSTEM.OBJ.Load("/path/to/Loader.cls", "ck")
- Invocation: Set sc = ##class(SNOMED.Loader).LoadConcepts("/path/to/sct2_Concept_Snapshot_*.txt")
- Copy class file into container: docker cp src/SNOMED/Loader.cls iris-ai-hub:/tmp/snomed/
- Compile : pipe $SYSTEM.OBJ.Load command to iris session
- Create test fixtures : Small 5-row RF2 files (concept + description) with known data
- Load and verify : Call loader methods, then SELECT COUNT(*) and spot-check specific rows
- Performance test : If full UK Monolith data is available, load and measure time
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md#3. ObjectScript vs Python Trade-offs — Lines 221-251, 274-281]
- [Source: _bmad-output/planning-artifacts/epics.md#Story 1.2]
- [Source: src/rf2.rs — RF2 column positions and file naming patterns]
- [Source: _bmad-output/implementation-artifacts/1-1-snomed-ct-data-model.md — Schema established, classes compiled]

Files changed

- src/SNOMED/Loader.cls (NEW)
- tests/fixtures/rf2/sct2_Concept_Snapshot_Test.txt (NEW)
- tests/fixtures/rf2/sct2_Description_Snapshot_Test.txt (NEW)

1-3-rf2-relationship-loader

Status: review | Source: 1-3-rf2-relationship-loader.html

User story

As a terminology administrator, I want to load SNOMED CT relationship RF2 files into IRIS, so that hierarchical and associative relationships are available for querying.

Acceptance criteria

1. SNOMED.Loader.LoadRelationships(filepath) inserts all rows from an RF2 relationship file into SNOMED.Relationship via SQL prepared statements
2. Processing is streamed line-by-line (no full dataset in memory)
3. Progress is reported every 50,000 records
4. Only active IS-A relationships (typeId = 116680003) are flagged for hierarchy use (printed in summary)
5. Querying SNOMED.Relationship by SourceId or DestinationId returns results efficiently via indexed lookups
6. Invalid or missing filepath returns a meaningful error status
7. TruncateAll is updated to also truncate the Relationship table

Implementation completion notes

- Added LoadRelationships method to existing SNOMED.Loader class
- Maps 6 RF2 columns (RelationshipId, SourceId, DestinationId, TypeId, Active, RelationshipGroup)
- Tracks active IS-A relationships (typeId=116680003) and reports count in summary
- Updated TruncateAll to include SNOMED.Relationship table
- Verified in Docker: 5 rows loaded (3 IS-A: 2 active + 1 inactive, 2 non-IS-A) IS-A count correctly reported as 2 (only active ones) SourceId indexed lookup returns correct results (46635009 → 2 relationships) TruncateAll clears all 3 tables
- 5 rows loaded (3 IS-A: 2 active + 1 inactive, 2 non-IS-A)
- IS-A count correctly reported as 2 (only active ones)
- SourceId indexed lookup returns correct results (46635009 → 2 relationships)
- TruncateAll clears all 3 tables
- src/SNOMED/Loader.cls (MODIFIED — added LoadRelationships, updated TruncateAll)
- tests/fixtures/rf2/sct2_Relationship_Snapshot_Test.txt (NEW)

Critical implementation notes

- ALL relationships are loaded (not just IS-A) — the full relationship table is needed for non-hierarchical queries too
- IS-A tracking is informational — print the count in summary, used by Story 2.2 (Transitive Closure) to know how many hierarchy edges exist
- relationshipGroup is an integer — convert with int() before passing to SQL
- Same patterns as Story 1.2 — streaming, prepared statements, error handling, progress reporting
- Update TruncateAll — must include Relationship table so full re-loads work cleanly
- [Source: _bmad-output/planning-artifacts/epics.md#Story 1.3]
- [Source: src/rf2.rs lines 232-249 — RF2 relationship column positions]
- [Source: src/SNOMED/Loader.cls — existing class to modify]

- [Source: src/SNOMED/Relationship.cls — target table schema]

Files changed

- src/SNOMED/Loader.cls (MODIFIED — added LoadRelationships, updated TruncateAll)
- tests/fixtures/rf2/sct2_Relationship_Snapshot_Test.txt (NEW)

2-1-full-text-lexical-search

Status: review | Source: 2-1-full-text-lexical-search.html

User story

As a terminology user, I want to search for SNOMED concepts by keyword or phrase across description terms, so that I can find relevant concepts without knowing their exact identifiers.

Acceptance criteria

1. A text search index exists on SNOMED.Description.Term enabling efficient full-text queries
2. SNOMED.Search.Lexical(term, maxResults) returns matching active descriptions with ConceptId, Term, and TypeId
3. Results include only active concepts (joined to SNOMED.Concept.Active = 1)
4. Results are ordered by relevance (exact matches first, then partial matches)
5. Search performs efficiently against 1.5M+ descriptions
6. Default maxResults is 25; caller can override

Implementation completion notes

- Added %iFind.Index.Basic index on Description.Term — compiles to helper class SNOMED.Description.tIDkRw
- Created SNOMED.Search class with ObjectScript Lexical(term, maxResults) method
- iFind query uses d.%ID %FIND search_index(TermIdx, ?, 0, '*') — the correct parameterized syntax for iFind
- JOIN to SNOMED.Concept ensures inactive concepts excluded (c.Active = 1)
- Results ordered: exact match first (CASE WHEN d.Term = ? THEN 0 ELSE 1 END), then alphabetical
- Returns valid JSON array: [{"ConceptId":"...","Term":"...","TypeId":"..."},...]
- Verified: “diabetes” returns 2 results (73211009, 46635009); inactive concept 404684003 excluded
- src/SNOMED/Description.cls (MODIFIED — added TermIdx iFind index)
- src/SNOMED/Search.cls (NEW)

Critical implementation notes

- iFind index type is %iFind.Index.Basic — NOT %iFind.Index.Analytic (which adds stemming/thesaurus complexity we don’t need)
- Return type is %String containing JSON — this is the standard pattern for methods that will become MCP tools (Story 3.1)
- JOIN is required — must filter by SNOMED.Concept.Active = 1 to exclude inactive concepts, not just inactive descriptions
- Both Active checks — d.Active = 1 AND c.Active = 1 (a description can be active on an inactive concept)
- %CONTAINSTERM matches individual words; for multi-word queries, %CONTAINS with the full phrase will match descriptions containing all words
- TOP clause — use SELECT TOP ? with maxResults parameter for efficient limiting
- No %New() / %Save() needed — this is a read-only query class
- ObjectScript string concatenation uses _ operator
- Do NOT use SQL LIKE '%term%' — this bypasses the text index and does a full table scan
- Do NOT use Python for this method — ObjectScript is correct for runtime query methods
- Do NOT add the iFind index to ConceptId or other non-text fields
- Do NOT return raw SQL result sets — return JSON string for MCP tool compatibility

- Do NOT forget the Concept.Active JOIN — inactive concepts must be excluded even if their descriptions are active
- Do NOT use %iFind.Index.Analytic — it's heavier and not needed for keyword matching
- Modified file: src/SNOMED/Description.cls (add iFind index)
- New file: src/SNOMED/Search.cls
- Package: SNOMED
- Target namespace: USER
- Reload test fixtures (TruncateAll + LoadConcepts + LoadDescriptions from Story 1.2 fixtures)
- Test fixture has concept 404684003 with Active=0 — searching for its description term should NOT return it
- Search for "diabetes" should return concepts 73211009 and 46635009
- Verify JSON output is parseable
- [Source: _bmad-output/planning-artifacts/epics.md#Story 2.1]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — line 503: "IRIS SQL + iFind/BM25 text index"]
- [Source: src/commands/lexical.rs — existing Rust lexical search for feature parity reference]
- [Source: src/SNOMED/Description.cls — target for iFind index addition]

Files changed

- src/SNOMED/Description.cls (MODIFIED — added TermIdx iFind index)
- src/SNOMED/Search.cls (NEW)

2-2-transitive-closure-computation

Status: review | Source: 2-2-transitive-closure-computation.html

User story

As a terminology administrator, I want to compute and store the transitive closure of SNOMED's IS-A hierarchy, so that ancestor/descendant queries and subsumption checks are instantaneous.

Acceptance criteria

1. SNOMED.TransitiveClosure.Compute() populates ^SNOMED.TC(ancestorId, descendantId) for all transitive IS-A paths
2. ^SNOMED.Children(parentId, childId) is populated with direct parent-child relationships
3. Computation handles the full SNOMED hierarchy (~831K concepts) without running out of memory
4. Progress is reported during computation
5. $DATA(^SNOMED.TC("73211009", "46635009")) returns true (Diabetes mellitus -> Type 1 diabetes)
6. A Clear() method kills the globals for re-computation

Implementation completion notes

- Created SNOMED.TransitiveClosure class with Python Compute() and Clear() methods
- Compute() uses iris.sql.prepare() to load active IS-A edges, iterates result set with for row in rs:
- BFS upward from each concept using collections.deque ; stores ancestors in ^SNOMED.TC(ancestor, descendant)
- Direct parent-child edges stored in ^SNOMED.Children(parent, child)
- Progress reporting every 10,000 concepts (not triggered in test with only 2 concepts)
- Clear() uses gref.kill([]) to kill both globals
- Verified all acceptance criteria: ^SNOMED.Children("73211009","46635009") exists (direct child) ^SNOMED.TC("73211009","46635009") exists (DM ancestor of Type 1 DM) ^SNOMED.TC("138875005","46635009") exists (transitive: root ancestor of Type 1 DM) ^SNOMED.TC("46635009","73211009") does NOT exist (wrong direction) ^SNOMED.TC("138875005","404684003") does NOT exist (inactive IS-A excluded)
- ^SNOMED.Children("73211009","46635009") exists (direct child)
- ^SNOMED.TC("73211009","46635009") exists (DM ancestor of Type 1 DM)
- ^SNOMED.TC("138875005","46635009") exists (transitive: root ancestor of Type 1 DM)
- ^SNOMED.TC("46635009","73211009") does NOT exist (wrong direction)
- ^SNOMED.TC("138875005","404684003") does NOT exist (inactive IS-A excluded)
- src/SNOMED/TransitiveClosure.cls (NEW)

Critical implementation notes

- Global subscript order is (ancestorId, descendantId) — this enables $ORDER(^SNOMED.TC(ancestorId, "")) to iterate all descendants of an ancestor
- IS-A relationship direction: In RF2, SourceId IS-A DestinationId (source is the child, destination is the parent)
- Only active IS-A: Filter TypeId = '116680003' AND Active = 1
- iris.gref() for global access — this is the correct Python API for setting global nodes
- Empty string value ("") — we only need existence, not a stored value
- BFS not DFS — BFS gives shortest path first (matching the Rust implementation)
- Memory: The parents_of dict for 831K concepts with ~1M edges is ~50-100MB — fits comfortably

- No self-referential entries — don’t store ^SNOMED.TC(X, X)
- Do NOT use iris.sql.prepare() for global writes — use iris.gref() directly (faster, no SQL overhead)
- Do NOT store the TC in a SQL table — globals are the correct architecture (O(1) via $DATA)
- Do NOT use networkx or other graph libraries — simple BFS with a deque is sufficient and avoids dependencies
- Do NOT compute from scratch if globals exist — the Clear() method should be called first
- Do NOT reverse the subscript order — it must be (ancestor, descendant) for efficient descendant iteration
- Do NOT include inactive IS-A relationships — only Active = 1
- New file: src/SNOMED/TransitiveClosure.cls
- Package: SNOMED
- Globals created: ^SNOMED.TC , ^SNOMED.Children
- iris.gref() API: gref = iris.gref("^GlobalName") then gref[subscript1, subscript2] = value
- Docker container: iris-ai-hub
- Testing: pipe commands to iris session IRIS -U USER
- ObjectScript global check: Write $DATA(^SNOMED.TC("73211009","46635009"))
- [Source: _bmad-output/planning-artifacts/epics.md#Story 2.2]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 65, 253-268, 508]
- [Source: src/commands/tct.rs — BFS algorithm reference from Rust implementation]
- [Source: tests/fixtures/rf2/sct2_Relationship_Snapshot_Test.txt — test data with IS-A edges]

Files changed

- src/SNOMED/TransitiveClosure.cls (NEW)

2-3-hierarchy-navigation-methods

Status: review | Source: 2-3-hierarchy-navigation-methods.html

User story

As a terminology user, I want to query ancestors, descendants, and check subsumption for any SNOMED concept, so that I can navigate the terminology hierarchy programmatically.

Acceptance criteria

1. SNOMED.Hierarchy.IsDescendantOf(conceptId, ancestorId) returns true/false in O(1) via global lookup
2. SNOMED.Hierarchy.GetAncestors(conceptId) returns all ancestor concept IDs as a JSON array
3. SNOMED.Hierarchy.GetDescendants(conceptId) returns all descendant concept IDs as a JSON array
4. SNOMED.Hierarchy.GetChildren(conceptId) returns only direct children (from ^SNOMED.Children)
5. SNOMED.Hierarchy.GetParents(conceptId) returns only direct parents

Implementation completion notes

- Created SNOMED.Hierarchy class with 5 ObjectScript classmethods
- IsDescendantOf — O(1) via $DATA(^SNOMED.TC(ancestorId, conceptId))#2
- GetDescendants — $ORDER iteration over ^SNOMED.TC(conceptId, desc)
- GetAncestors — scans all first-level subscripts of ^SNOMED.TC , checks $DATA for conceptId as descendant
- GetChildren — $ORDER over ^SNOMED.Children(conceptId, child)
- GetParents — scans first-level subscripts of ^SNOMED.Children , checks $DATA for conceptId as child
- All methods return JSON strings (via %DynamicArray.%ToJSON()) except IsDescendantOf which returns %Boolean
- All acceptance criteria verified in Docker with test fixture data
- src/SNOMED/Hierarchy.cls (NEW)

Critical implementation notes

- Return type is %String containing JSON — matches the pattern from SNOMED.Search.Lexical() for MCP tool compatibility (Story 3.2)
- IsDescendantOf returns %Boolean — the only method that returns a scalar (not JSON)
- $DATA(...)#2 — the #2 ensures we get 1 only if the node itself has data (not just child nodes)
- $ORDER(^Global(subscript)) pattern — iterates next subscript at that level; returns “” when exhausted
- ObjectScript string concatenation uses _ operator
- %DynamicArray literal [] creates a JSON array; .%Push() appends, .%ToJSON() serializes
- No Python — all methods are pure ObjectScript for performance
- All methods handle missing concept gracefully — if concept not in globals, returns empty array or 0
- Do NOT use Python for these methods — ObjectScript $ORDER / $DATA is native and fastest
- Do NOT create a reverse index global — the scan approach is acceptable for these utility methods
- Do NOT return %DynamicArray objects — return JSON strings for MCP compatibility
- Do NOT forget #2 on $DATA checks — without it, nodes with children but no data return non-zero
- Do NOT add error handling for missing globals — empty iteration naturally returns empty results
- Do NOT store results in a SQL table — these are runtime query methods against globals

- GetDescendants and GetChildren are O(k) where k = number of results (direct subscript iteration)
- GetAncestors and GetParents are O(n) where n = number of first-level subscripts in global (must scan all ancestors/parents to find matching ones)
- For the full SNOMED CT (~831K concepts), GetAncestors scans up to ~400K distinct ancestor entries — this is acceptable for interactive use but could be slow for batch operations
- IsDescendantOf is always O(1) — single $DATA lookup
- New file: src/SNOMED/Hierarchy.cls
- Package: SNOMED
- Dependencies: ^SNOMED.TC and ^SNOMED.Children globals (from Story 2.2)
- [Source: _bmad-output/planning-artifacts/epics.md#Story 2.3]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 253-271: ObjectScript hierarchy traversal patterns]
- [Source: src/SNOMED/TransitiveClosure.cls — global structure this story queries]
- [Source: src/SNOMED/Search.cls — ObjectScript JSON return pattern to follow]

Files changed

- src/SNOMED/Hierarchy.cls (NEW)

3-1-snomed-lookup-and-search-tools

Status: review | Source: 3-1-snomed-lookup-and-search-tools.html

User story

As a Claude Desktop/Code user, I want MCP tools that let me look up SNOMED concepts by ID and search by term, so that I can query terminology through natural language conversation.

Acceptance criteria

1. SNOMED.Tools.Lookup extends %AI.Tool with a GetConcept(sctid) method that returns JSON with concept details (ConceptId, FSN, Active, descriptions list)
2. SNOMED.Tools.Search extends %AI.Tool with a LexicalSearch(term, maxResults) method that returns matching active concepts
3. Results are limited to maxResults (default 20) and only include active concepts
4. A Query-as-Tool SearchByPattern is defined as a class query returning matches in the standard {rows, row_count, truncated} envelope
5. Tools compile successfully and methods return correct JSON when called directly

Implementation completion notes

- Created SNOMED.Tools.Lookup extending %AI.Tool with GetConcept(sctid) method Queries Concept table for metadata + Description table for active descriptions Identifies FSN via TypeId 900000000000003001 Returns clean error JSON for not-found concepts
- Queries Concept table for metadata + Description table for active descriptions
- Identifies FSN via TypeId 900000000000003001
- Returns clean error JSON for not-found concepts
- Created SNOMED.Tools.Search extending %AI.Tool with: LexicalSearch(term, maxResults=20) — thin wrapper delegating to SNOMED.Search.Lexical() SearchByPattern class query — SQL LIKE pattern matching as Query-as-Tool
- LexicalSearch(term, maxResults=20) — thin wrapper delegating to SNOMED.Search.Lexical()
- SearchByPattern class query — SQL LIKE pattern matching as Query-as-Tool
- Both classes compile cleanly and extend %AI.Tool for MCP discovery
- All acceptance criteria verified: GetConcept returns FSN, Active, Descriptions array Not-found returns {"error":"Concept not found","sctid":"..."} LexicalSearch returns 2 diabetes results (excludes inactive concept) SearchByPattern query exists and underlying SQL returns correct results
- GetConcept returns FSN, Active, Descriptions array
- Not-found returns {"error":"Concept not found","sctid":"..."}
- LexicalSearch returns 2 diabetes results (excludes inactive concept)
- SearchByPattern query exists and underlying SQL returns correct results
- src/SNOMED/Tools/Lookup.cls (NEW)
- src/SNOMED/Tools/Search.cls (NEW)

Critical implementation notes

- Must extend %AI.Tool — this is what makes methods discoverable as MCP tools
- ObjectScript only — %AI.Tool methods must be ObjectScript, not Python
- Return %String with JSON — tool results are serialized JSON

- Doc comments (///) are tool descriptions — write them clearly for LLM consumption
- FSN TypeId is 900000000000003001 — Fully Specified Name type identifier in SNOMED
- Synonym TypeId is 900000000000013009 — for reference
- SearchByPattern uses SQL LIKE — NOT iFind (LIKE supports wildcards the LLM can construct)
- Query SqlProc annotation — required for the query to be callable as a stored procedure
- Do NOT re-implement search logic — delegate to SNOMED.Search.Lexical()
- Do NOT use Python for tool methods — ObjectScript is required by %AI.Tool
- Do NOT return raw SQL result sets — always serialize to JSON
- Do NOT forget the /// doc comments — they ARE the tool descriptions
- Do NOT put tools in src/SNOMED/Search.cls or existing files — create new files in src/SNOMED/Tools/
- Do NOT add ToolSet or MCP Service yet — that's Story 3.3
- [Source: _bmad-output/planning-artifacts/epics.md#Story 3.1]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 99-131: Tool Registration Pattern]
- [Source: src/SNOMED/Search.cls — existing lexical search to delegate to]
- [Source: src/SNOMED/Hierarchy.cls — pattern reference for ObjectScript JSON methods]

Files changed

- src/SNOMED/Tools/Lookup.cls (NEW)
- src/SNOMED/Tools/Search.cls (NEW)

3-2-snomed-hierarchy-tools

Status: review | Source: 3-2-snomed-hierarchy-tools.html

User story

As a Claude Desktop/Code user, I want MCP tools for navigating the SNOMED hierarchy (ancestors, descendants, subsumption), so that I can explore clinical concept relationships through conversation.

Acceptance criteria

1. SNOMED.Tools.Hierarchy extends %AI.Tool with hierarchy navigation methods
2. IsA(conceptId, ancestorId) returns true/false indicating subsumption relationship
3. GetAncestors(conceptId) returns a JSON array of ancestor concepts with their preferred terms
4. GetDescendants(conceptId, maxResults) returns descendant concepts (limited to maxResults , default 100) with preferred terms
5. GetChildren(conceptId) returns only direct child concepts with preferred terms
6. GetParents(conceptId) returns only direct parent concepts with preferred terms

Implementation completion notes

- Created SNOMED.Tools.Hierarchy extending %AI.Tool with 5 methods
- IsA returns JSON {"isDescendant":1/0,"conceptId":"...","ancestorId":"..."}
- GetAncestors , GetChildren , GetParents delegate to SNOMED.Hierarchy then enrich via %DynamicArray.%FromJSON() + %GetIterator()
- GetDescendants iterates ^SNOMED.TC global directly with maxResults limit (default 100)
- All methods enrich concept IDs with FSN (TypeId 900000000000003001) via prepared SQL statement reuse
- Verified all 6 acceptance criteria with test fixture data
- src/SNOMED/Tools/Hierarchy.cls (NEW)

Critical implementation notes

- Extend %AI.Tool — required for MCP tool discovery
- Return %String with JSON — all tool methods must return JSON strings
- Enrich with FSN — tool methods return human-readable terms, not just IDs
- FSN TypeId = 900000000000003001 — Fully Specified Name
- Reuse prepared statements — prepare SQL once, execute multiple times within a method
- maxResults default = 100 for GetDescendants — prevents overwhelming the LLM with huge result sets
- %DynamicArray.%FromJSON() to parse JSON strings back into iterable arrays
- %GetIterator() + %GetNext(.key, .value) to iterate %DynamicArray objects
- Doc comments (///) on methods — these become LLM-visible tool descriptions
- GetDescendants iterates global directly — avoids loading all descendants into memory just to limit
- Do NOT return raw boolean from IsA — return JSON for consistency
- Do NOT load all descendants then slice — iterate global with a counter limit
- Do NOT forget to handle concepts without FSN — set term to empty string
- Do NOT create a new Hierarchy class — extend the EXISTING SNOMED.Hierarchy via delegation

- Do NOT put this in src/SNOMED/Hierarchy.cls — create NEW file src/SNOMED/Tools/Hierarchy.cls
- Do NOT skip enrichment — the LLM needs readable concept names, not just SCTIDs
- [Source: _bmad-output/planning-artifacts/epics.md#Story 3.2]
- [Source: src/SNOMED/Tools/Lookup.cls — %AI.Tool pattern reference]
- [Source: src/SNOMED/Hierarchy.cls — delegation target for hierarchy queries]

Files changed

- src/SNOMED/Tools/Hierarchy.cls (NEW)

3-3-mcp-service-and-server-configuration

Status: review | Source: 3-3-mcp-service-and-server-configuration.html

User story

As a Claude Desktop/Code user, I want the SNOMED tools registered as an MCP service accessible via iris-mcp-server , so that I can use SNOMED tools directly from my AI assistant.

Acceptance criteria

1. SNOMED.ToolSet extends %AI.ToolSet with XData Definition including all tool classes (Lookup, Search, Hierarchy)
2. SNOMED.MCP.Service extends %AI.MCP.Service with Parameter SPECIFICATION = "SNOMED.ToolSet"
3. Both classes compile successfully in the IRIS container
4. A config.toml template is provided for iris-mcp-server stdio transport pointing to /mcp/snomed
5. A Claude Desktop config snippet is documented for connecting to the SNOMED MCP service

Implementation completion notes

- Created SNOMED.ToolSet extending %AI.ToolSet with XData composing 3 tool classes
- Created SNOMED.MCP.Service extending %AI.MCP.Service with SPECIFICATION = “SNOMED.ToolSet”
- Created config/mcp-config.toml with stdio transport, connection pool, and /mcp/snomed endpoint
- ToolSet %Discover() returns all 8 tools: GetConcept, LexicalSearch, SearchByPattern, IsA, GetAncestors, GetDescendants, GetChildren, GetParents
- Each tool has proper JSON Schema parameters, descriptions from /// comments, and return types
- Both classes compile cleanly in IRIS 2026.2.0AI
- src/SNOMED/ToolSet.cls (NEW)
- src/SNOMED/MCP/Service.cls (NEW)
- config/mcp-config.toml (NEW)

Critical implementation notes

- ToolSet XData MUST be application/xml — the MimeType attribute is required
- <Include Class="..."/> references full class names — including package
- SPECIFICATION parameter is a STRING — set to the full class name of the ToolSet
- File structure follows research architecture: src/SNOMED/ToolSet.cls src/SNOMED/MCP/Service.cls config/mcp-config.toml
- src/SNOMED/ToolSet.cls
- src/SNOMED/MCP/Service.cls
- config/mcp-config.toml
- No runtime testing of MCP server — iris-mcp-server binary may not be in the container; just verify compilation
- Do NOT try to run iris-mcp-server in the container — it may not be installed
- Do NOT try to register the web application programmatically — that’s deployment, not development
- Do NOT include credentials in the config template that aren’t clearly marked as examples
- Do NOT modify any existing Tool classes — this story only creates ToolSet, Service, and config
- Do NOT use SpecificationClass parameter — use SPECIFICATION (matches research doc pattern)
- [Source: _bmad-output/planning-artifacts/epics.md#Story 3.3]

- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 99-166: Tool Registration + config.toml]
- [Source: src/SNOMED/Tools/*.cls — tool classes to compose into ToolSet]

Files changed

- src/SNOMED/ToolSet.cls (NEW)
- src/SNOMED/MCP/Service.cls (NEW)
- config/mcp-config.toml (NEW)

4-1-embedding-storage-and-vector-schema

Status: review | Source: 4-1-embedding-storage-and-vector-schema.html

User story

As a developer, I want a persistent class with a VECTOR column to store concept description embeddings, so that semantic similarity search can be performed via SQL.

Acceptance criteria

1. SNOMED.ConceptEmbedding class extends %Persistent with properties: ConceptId (string, indexed), Term (string), and Embedding (VECTOR(DOUBLE, 384))
2. An ANN index (HNSW) is defined on the Embedding column for efficient similarity search
3. The class compiles successfully in the IRIS container
4. An embedding vector can be inserted via SQL INSERT INTO SNOMED.ConceptEmbedding (ConceptId, Term, Embedding) VALUES (?, ?, TO_VECTOR(?))
5. Inserted vectors are queryable via VECTOR_COSINE() similarity function

Implementation completion notes

- Created SNOMED.ConceptEmbedding extending %Persistent with ConceptId, Term, and Embedding (VECTOR DOUBLE 384) properties
- ConceptId has unique index for fast lookup
- HNSW ANN index defined using As %SQL.Index.HNSW syntax
- Verified INSERT with TO_VECTOR(?, DOUBLE) works
- Verified VECTOR_COSINE() returns correct similarity scores (1.0 same, 0 orthogonal)
- Verified ConceptId index lookup works
- All 5 acceptance criteria satisfied
- src/SNOMED/ConceptEmbedding.cls (NEW)

Critical implementation notes

- Dimension = 384 — matches MiniLM/nomic-embed-text output size
- DOUBLE datatype — not FLOAT; research docs specify DOUBLE
- ConceptId is unique indexed — one embedding per concept (FSN only)
- Term stores the text that was embedded — so search results can show the original text
- ANN index is HNSW — required for performance at 831K+ vectors
- HNSW parameters : m=16, efConstruction=200 (standard defaults for quality/speed balance)
- File location : src/SNOMED/ConceptEmbedding.cls
- Test with fake vectors — don’t need real embeddings to verify schema works
- Do NOT implement embedding generation — that’s Story 4.2
- Do NOT implement the semantic search method — that’s Story 4.3
- Do NOT add Python dependencies (ollama, sentence-transformers) — not needed for schema
- Do NOT modify existing classes (Concept, Description, etc.)
- Do NOT add the ConceptEmbedding to the ToolSet — it’s storage only, not a tool

- Do NOT try to create a foreign key to SNOMED.Concept — keep it simple; just indexed ConceptId string
- [Source: _bmad-output/planning-artifacts/epics.md#Story 4.1]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 285-370: Vector/Embedding Support]
- [Source: src/SNOMED/Concept.cls — %Persistent class pattern reference]

Files changed

- src/SNOMED/ConceptEmbedding.cls (NEW)

4-2-embedding-generation-pipeline

Status: review | Source: 4-2-embedding-generation-pipeline.html

User story

As a terminology administrator, I want to generate vector embeddings for all active SNOMED concept descriptions using Ollama, so that the terminology is searchable by semantic meaning.

Acceptance criteria

1. SNOMED.Embeddings.Generate(modelName, batchSize) generates embeddings for all active Fully Specified Names
2. Processing is batched (default batch size 100) to manage memory
3. Progress is reported (count and percentage)
4. Embeddings are inserted into SNOMED.ConceptEmbedding via SQL prepared statements
5. The process can be resumed (skips concepts that already have embeddings)
6. If Ollama is unreachable or returns an error, a meaningful error is returned and partial progress is preserved

Implementation completion notes

- Created SNOMED.Embeddings with Generate(modelName, batchSize, ollamaUrl) Python class method
- Resume support via NOT IN (SELECT ConceptId FROM SNOMED.ConceptEmbedding) subquery
- Batched processing with progress reporting (count/total/percentage per batch)
- Comprehensive error handling: ConnectionError, InvalidURL, Timeout, HTTPError, RequestException, mismatched embedding count
- Partial progress preserved on failure (no transaction wrapping)
- Verified: resume logic (1 pre-embedded, 3 remaining = correct), nothing-to-do path, connection error returns error status
- src/SNOMED/Embeddings.cls (NEW)

Critical implementation notes

- Model default = “all-minilm” — 384 dimensions, matches VECTOR column
- Batch size default = 100 — balance between speed and memory
- FSN TypeId = ‘900000000000003001’ — only embed Fully Specified Names
- Only active concepts/descriptions — filter on Active = 1 for both
- TO_VECTOR(?, DOUBLE) — the DOUBLE qualifier is REQUIRED (confirmed in Story 4.1)
- Vector as comma-separated string — join floats with comma for TO_VECTOR()
- Progress reporting every batch — print count, total, percentage
- Partial progress preserved — no transaction wrapping; each INSERT commits immediately
- File location : src/SNOMED/Embeddings.cls
- Class extends %RegisteredObject — not %Persistent ; it’s a utility class
- Do NOT use sentence_transformers or local model loading — use Ollama HTTP API
- Do NOT wrap the entire operation in a transaction — partial progress must persist
- Do NOT modify SNOMED.ConceptEmbedding class — it’s complete from Story 4.1
- Do NOT implement semantic search — that’s Story 4.3
- Do NOT hardcode Ollama URL — accept as parameter with default

- Do NOT embed Synonyms — only FSN (one embedding per concept)
- Do NOT assume requests is installed — check and fallback to urllib
- Do NOT add a Clear() method that would delete all embeddings — too dangerous for accidental calls
- Compile test : Verify class compiles without errors
- Resume logic test : Insert a few fake embeddings, run Generate, verify it skips those
- Connection error test : Call with bad URL, verify error message and status
- If Ollama available : Test with 2-3 real concepts and verify embeddings stored
- [Source: _bmad-output/planning-artifacts/epics.md#Story 4.2]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 352-370: Embedding Generation Strategy]
- [Source: src/SNOMED/Loader.cls — Python bulk operation pattern]
- [Source: src/SNOMED/ConceptEmbedding.cls — target table schema]

Files changed

- src/SNOMED/Embeddings.cls (NEW)

4-3-semantic-search-method-and-mcp-tool

Status: review | Source: 4-3-semantic-search-method-and-mcp-tool.html

User story

As a Claude Desktop/Code user, I want to search for SNOMED concepts by semantic meaning through an MCP tool, so that I can find relevant concepts even when I don’t know the exact clinical terminology.

Acceptance criteria

1. SNOMED.Search.Semantic(query, maxResults) embeds the query via Ollama, then uses VECTOR_COSINE() to return the top N matching concepts
2. Results include ConceptId, Term, and similarity score
3. Only results above a minimum threshold (default 0.5) are returned
4. SNOMED.Tools.Semantic extends %AI.Tool with a SemanticSearch(query, maxResults) method
5. The ToolSet XData is updated to include the Semantic tool class
6. The tool is discoverable and callable via the MCP service

Implementation completion notes

- Added Semantic() Python classmethod to existing SNOMED.Search class
- Created SNOMED.Tools.Semantic extending %AI.Tool with SemanticSearch(query, maxResults) method
- Updated SNOMED.ToolSet XData to include SNOMED.Tools.Semantic
- ToolSet now discovers 9 tools (was 8): GetConcept, LexicalSearch, SearchByPattern, GetAncestors, GetChildren, GetDescendants, GetParents, IsA, SemanticSearch
- Verified: error JSON on Ollama connection failure, vector similarity scoring with threshold filtering, tool delegation chain
- src/SNOMED/Search.cls (MODIFIED — added Semantic method)
- src/SNOMED/Tools/Semantic.cls (NEW)
- src/SNOMED/ToolSet.cls (MODIFIED — added Include for Semantic)

Critical implementation notes

- Add Semantic to EXISTING SNOMED.Search class — do NOT create a new class for the search method
- SNOMED.Tools.Semantic is a NEW file at src/SNOMED/Tools/Semantic.cls
- Tool delegates to Search — SNOMED.Tools.Semantic.SemanticSearch() calls ##class(SNOMED.Search).Semantic()
- Default minScore = 0.5 — filter out low-quality matches
- Default maxResults = 10 — keep result sets manageable for LLM
- Default model = “all-minilm” — must match the model used for embedding generation (Story 4.2)
- requests is available in the container (confirmed in Story 4.2, version 2.33.1)
- Return JSON string from ALL tool methods — never raw values
- /// doc comments become tool descriptions — write them clearly for the LLM
- ToolSet uses <Include Class="SNOMED.Tools.Semantic"/> XML — add after Hierarchy
- Do NOT create a separate SNOMED.Search.Semantic class — add method to existing SNOMED.Search
- Do NOT use sentence_transformers — use Ollama HTTP API (same as Story 4.2)
- Do NOT return results below minScore threshold — filter them out

- Do NOT hardcode the Ollama URL in the Tool class — pass through parameters
- Do NOT modify the MCP.Service class — it auto-discovers from ToolSet
- Do NOT change how existing tools work — only ADD new functionality
- Do NOT forget to update ToolSet XData — or the tool won't be discoverable
- Compile all classes — Search.cls (modified), Tools/Semantic.cls (new), ToolSet.cls (modified)
- ToolSet discovery — call %Discover() and verify SemanticSearch appears
- Error path — call SNOMED.Search.Semantic("test", 10, 0.5, "http://localhost:59999") with bad URL
- If Ollama available — insert fake embeddings, embed a query, verify results with scores
- [Source: _bmad-output/planning-artifacts/epics.md#Story 4.3]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 296-325: SQL Vector Syntax]
- [Source: src/SNOMED/Search.cls — existing class to modify]
- [Source: src/SNOMED/Tools/Lookup.cls — %AI.Tool pattern reference]
- [Source: src/SNOMED/ToolSet.cls — XData to update]

Files changed

- src/SNOMED/Search.cls (MODIFIED — added Semantic method)
- src/SNOMED/Tools/Semantic.cls (NEW)
- src/SNOMED/ToolSet.cls (MODIFIED — added Include for Semantic)

5-1-ipm-package-structure

Status: review | Source: 5-1-ipm-package-structure.html

User story

As a developer, I want the project structured as an IPM module with module.xml , so that it can be installed into any IRIS for Health instance via zpm "install snomed-iris" .

Acceptance criteria

1. A module.xml at the project root identifies package name (snomed-iris), version, description, and keywords
2. SourcesRoot points to the src/ directory containing all .cls files
3. A requirements.txt is included listing Python dependencies (e.g., requests)
4. An installer class SNOMED.Installer is invoked during the Configure phase
5. SNOMED.Installer.Setup() installs Python dependencies via pip
6. SNOMED.Installer.Setup() registers the MCP Server application /mcp/snomed programmatically
7. A success message confirms installation

Implementation completion notes

- Created module.xml at project root with IPM package metadata (name, version 0.4.0, description, keywords, SourcesRoot, Resource, Invoke)
- Created requirements.txt with single requests dependency
- Created src/SNOMED/Installer.cls with Setup() , InstallPythonDeps() (Python), and RegisterMCPApp() (ObjectScript)
- Installer gracefully handles missing requirements.txt, pip failures, and web app registration errors (warnings only, never fails install)
- All 7 acceptance criteria satisfied: module.xml identifies package, SourcesRoot points to src/, requirements.txt included, Installer invoked at Configure phase, pip install works, web app registered, success message printed
- module.xml (NEW)
- requirements.txt (NEW)
- src/SNOMED/Installer.cls (NEW)

Critical implementation notes

- module.xml at project ROOT — not in src/ or config/
- requirements.txt at project ROOT — next to module.xml
- Version = “0.4.0” — matches project status (Epics 1-4 complete)
- <SourcesRoot>src</SourcesRoot> — IPM loads all .cls files recursively from src/
- <Resource Name="SNOMED.PKG"/> — includes all classes in the SNOMED package
- Installer class at src/SNOMED/Installer.cls — so it gets compiled by IPM before invocation
- Do NOT fail on web app registration error — return OK with warning; deployment may need different auth
- Do NOT fail on pip install error — requests may already be installed; just warn
- New $Namespace before switching — preserve caller’s namespace
- Set $Namespace = "%SYS" — required for Security.Applications access
- Do NOT create a Dockerfile — that’s Story 5.2

- Do NOT create a README — that's Story 5.3
- Do NOT modify any existing source classes
- Do NOT add dependencies beyond requests — it's the only external Python package used
- Do NOT make the installer fail hard on non-critical errors (web app, pip)
- Do NOT include config/mcp-config.toml in module.xml — it's a template, not installable code
- Do NOT try to run zpm "install" — just verify module.xml is valid and Installer compiles/runs
- Verify module.xml is well-formed — parse as XML
- Compile Installer.cls — must compile without errors
- Run Setup() — should complete without throwing
- Verify web app exists — check Security.Applications.Exists("/mcp/snomed") after Setup
- [Source: _bmad-output/planning-artifacts/epics.md#Story 5.1]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 373-434: IPM Packaging]

Files changed

- module.xml (NEW)
- requirements.txt (NEW)
- src/SNOMED/Installer.cls (NEW)

5-2-docker-container-image

Status: review | Source: 5-2-docker-container-image.html

User story

As a DevOps engineer, I want a Dockerfile that builds a ready-to-run IRIS for Health container with SNOMED tools pre-installed, so that teams can deploy with docker run without manual setup.

Acceptance criteria

1. A Dockerfile based on containers.intersystems.com/intersystems/irishealth-community:2026.2.0AI builds successfully
2. The image installs the IPM package, compiles all classes, and configures the MCP server application
3. The iris-mcp-server binary and config/mcp-config.toml template are included
4. Ports 1972 (SuperServer) and 52773 (Web Portal) are exposed
5. docker run starts IRIS with the SNOMED namespace ready and /mcp/snomed accessible
6. A docker-compose.yml is provided showing Ollama sidecar configuration for embeddings

Implementation completion notes

- Created Dockerfile using irishealth-community:2026.2.0AI base image
- Uses separate iris-init.script for clean ObjectScript execution during build (avoids heredoc/inline issues)
- Build sequence: pip install → copy source → start IRIS → load classes + run Installer → stop IRIS → copy config
- Created docker-compose.yml with IRIS + Ollama sidecar services, persistent volumes, healthcheck
- Validated: Dockerfile parses correctly (docker build –check), docker-compose.yml validates cleanly
- Note: Full build test requires InterSystems container registry access (not available on this machine)
- All 6 acceptance criteria addressed
- Dockerfile (NEW)
- docker-compose.yml (NEW)
- iris-init.script (NEW)

Critical implementation notes

- Base image MUST be irishealth-community:2026.2.0AI — this is the AI Hub EAP image with %AI.Tool support
- Use USER irisowner before any IRIS operations — root cannot start IRIS
- Use USER root for system package installs (pip) — switch back to irisowner after
- $SYSTEM.OBJ.LoadDir(dir, "ck", .err, 1) — the 1 flag means recursive loading
- iris stop IRIS quietly — the quietly flag prevents the stop from writing to the journal
- Do NOT use zpm in Dockerfile — package is local, not published to a registry
- EXPOSE 1972 52773 — SuperServer for MCP (wgproto) and Web Portal for HTTP/REST
- Healthcheck with iris qlist — standard pattern for IRIS container health verification
- Do NOT include RF2 data in the image — users load their own SNOMED release at runtime
- Do NOT include Ollama in the IRIS image — it’s a separate sidecar service
- config/mcp-config.toml already exists — created in Story 3.3, just COPY it
- Do NOT publish the IPM package to any registry — this is a local build

- Do NOT include SNOMED RF2 data files in the Docker image
- Do NOT install Ollama inside the IRIS container — use docker-compose sidecar
- Do NOT modify any existing source classes (Installer, MCP.Service, ToolSet, etc.)
- Do NOT modify module.xml or requirements.txt
- Do NOT hardcode credentials beyond the development defaults (CSPSystem/SYS)
- Do NOT create a README — that's Story 5.3
- Do NOT use IRIS mirror or ECP configurations — single instance only
- Verify Dockerfile builds — docker build -t snomed-iris . should succeed
- Verify container starts — docker run -d snomed-iris → IRIS comes up healthy
- Verify classes compiled — exec into container and check $SYSTEM.OBJ.IsUpToDate("SNOMED.Concept")
- Verify web app registered — check Security.Applications.Exists("/mcp/snomed") returns true
- Verify docker-compose.yml — parse as valid YAML, both services defined
- Verify ports exposed — inspect image metadata for 1972 and 52773
- [Source: _bmad-output/planning-artifacts/epics.md#Story 5.2]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 436-448: Container Distribution]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 453-496: Architecture Diagram]
- [Source: config/mcp-config.toml — existing MCP server configuration]
- [Source: _bmad-output/implementation-artifacts/5-1-ipm-package-structure.md — previous story context]

Files changed

- Dockerfile (NEW)
- docker-compose.yml (NEW)
- iris-init.script (NEW)

5-3-documentation-and-getting-started-guide

Status: review | Source: 5-3-documentation-and-getting-started-guide.html

User story

As a new user, I want clear documentation on how to install, load data, and connect to Claude Desktop, so that I can get the SNOMED MCP tools working quickly.

Acceptance criteria

1. A README.md exists in the project root explaining prerequisites, installation, RF2 data loading, and MCP configuration
2. A quick-start section with copy-paste commands is included
3. All available MCP tools are documented with example queries
4. The config/mcp-config.toml template works with iris-mcp-server out of the box for local development

Implementation completion notes

- Added comprehensive “IRIS for Health AI Hub Edition” section to existing README.md
- Documents: prerequisites, Docker quick start, IPM install, RF2 loading, embedding generation, MCP configuration
- All 9 MCP tools documented in tables with parameters and example queries
- Updated config/mcp-config.toml with descriptive inline comments for all sections
- All acceptance criteria satisfied: README exists, quick-start included, tools documented, config template works
- README.md (MODIFIED — added IRIS AI Hub section)
- config/mcp-config.toml (MODIFIED — added inline comments)

Critical implementation notes

- Create README.md at project root — this is a NEW file
- Do NOT modify any .cls source files — documentation only
- Do NOT modify module.xml , Dockerfile , docker-compose.yml — only reference them
- Tool list MUST match actual ToolSet — exactly 9 tools as listed above
- config/mcp-config.toml already exists — only add/improve inline comments if needed
- Use GitHub-flavored markdown — tables, code blocks, headings
- Include BOTH Docker and IPM installation paths — Docker for quick start, IPM for existing IRIS instances
- Ollama is OPTIONAL — only needed for embedding generation and semantic search
- Do NOT include actual RF2 data or download links — users must obtain their own SNOMED license
- Do NOT create any new source code files
- Do NOT modify existing code
- Do NOT include copyrighted SNOMED data
- Do NOT add external badges that require service registration
- Do NOT write a CONTRIBUTING.md or LICENSE file (out of scope)
- Do NOT document internal implementation details (globals, SQL schema)
- Do NOT change config/mcp-config.toml structure — only add comments
- Verify README renders — valid markdown with no broken links

- Verify tool count — exactly 9 tools documented
- Verify file paths — all referenced files exist in the project
- Verify config template — config/mcp-config.toml has appropriate comments
- [Source: _bmad-output/planning-artifacts/epics.md#Story 5.3]
- [Source: _bmad-output/planning-artifacts/research/technical-iris-for-health-ai-hub-snomed-ct-port-research-2026-05-13.md — lines 136-154: config.toml]
- [Source: config/mcp-config.toml — existing MCP configuration template]
- [Source: _bmad-output/implementation-artifacts/5-2-docker-container-image.md — Docker context]
- [Source: _bmad-output/implementation-artifacts/5-1-ipm-package-structure.md — IPM context]

Files changed

- README.md (MODIFIED — added IRIS AI Hub section)
- config/mcp-config.toml (MODIFIED — added inline comments)